Report of U-Net semantic segmentation on LIP dataset

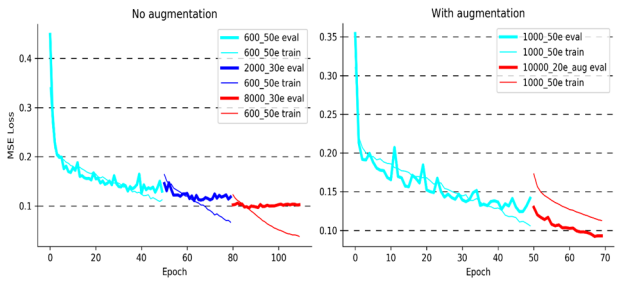
https://github.com/Brookekekeboom/CV-pytorch

Xuchen Gong

I Segment body from background

1.1 Segment single person

I trained U-Net on the database of LIP single person. Because of large size of the database, I trained the model on a small number of images first and continue training it on more images afterward, which can be illustrated as follows:

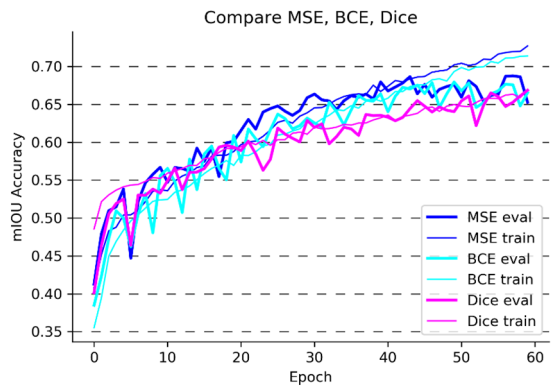


The highest IOU eval accuracy reached 76% after:
(1) train 1000 images for 50 epochs with MSE
(2) train 10000 images for 20 epochs with MSE and with Color Jitter added
(3) train another 10000 images for 20 epochs with Dice

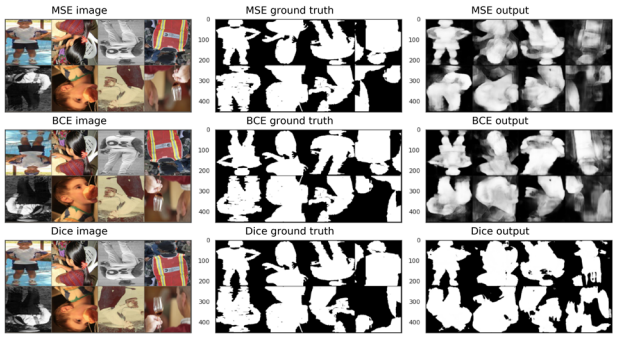
1.2 Experiments on MSE, BCE and Dice loss

I compared the performance of MSE loss, BCE loss and Dice loss.

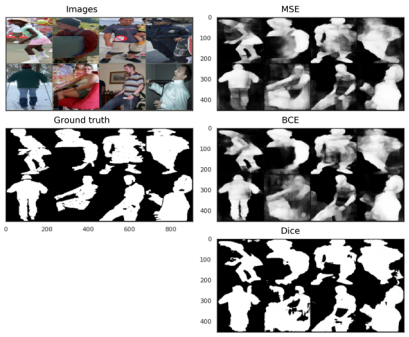
The results show that MSE and BCE give similar IOU eval acc and the output of BCE is slightly more confident than that of MSE. In terms of confidence of prediction, Dice > BCE > MSE. Moreover, Dice has a lower eval acc than MSE and BCE while it displays a much less severe tendency of overfitting at the end of the 60 epochs. In terms of overfitting, MSE ≈ BCE > Dice.



Demo on the training set:



Demo on the eval set:



1.3 Experiment on data augmentation

I experimented with several data augmentation techniques, finding that *random rotation* noticeably relieves overfitting while *ransom resize crop* does not. Moreover, *elastic deformation* with parameters alpha = 3.5 × image size and sigma = 0.08 × image size works well. In addition, among the parameters of *Color Jitter*, hue is not noticeably helpful for this task.

Other findings:
- U-Net with padding performs better than U-Net without padding, and a up conv of 1×1 performs better than one of 3×3.
- Using default initialization, it takes a long time for the output to get rid of blurry grey, while initializing the conv to normal (0, sqrt (2. / kernel_area / out_channels)) largely fastens the training.

II Segment different body parts

2.1 Segment 20 classes

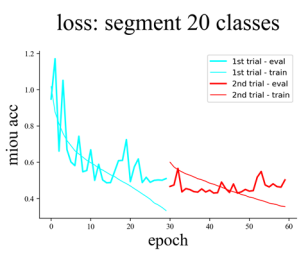
I used U-Net to segment 20 classes and the highest eval accuracy reached 27% after training U-Net on 20000 images for 60 epochs in total. Further training which involves more data augmentation and training images did not improve the accuracy.

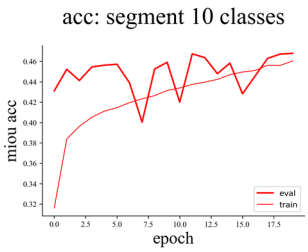
2.2 Segment 10 classes

Observing the model's failure to distinguish *LeftArm* from *RightArm*, *UpperClothes* from *Coat*, *Pants*, *LeftLeg* from *RightLeg*, and *LeftShoe* from *RightShoe*, I tried distributing the 19 classes (20 in total) into 9 classes (10 in total) and renumbering them to 0-10 as follows.

New label	Old label
1 Hat	1 Hat
2 Hair	2 Hair
3 Arms	3 Gloves, 14 LeftArm, 15 RightArm
4 Sunglasses	4 Sunglasses
5 UpperCloth	5 UpperCloth, 6 Dress, 7 Coat, 10 Jumpsuits, 11 Scarf
6 LowerCloth	8 Socks, 9 Pants, 16 LeftLeg, 17 RightLeg
7 Skirt	12 Skirt
8 Face	13 Face
9 Shoes	18 LeftShoe, 19 RightShoe

I segmented 10 classes by fine tuning the model trained for 20-class segmentation, and its eval accuracy reached 46%, around 20% higher than the 20-class one.





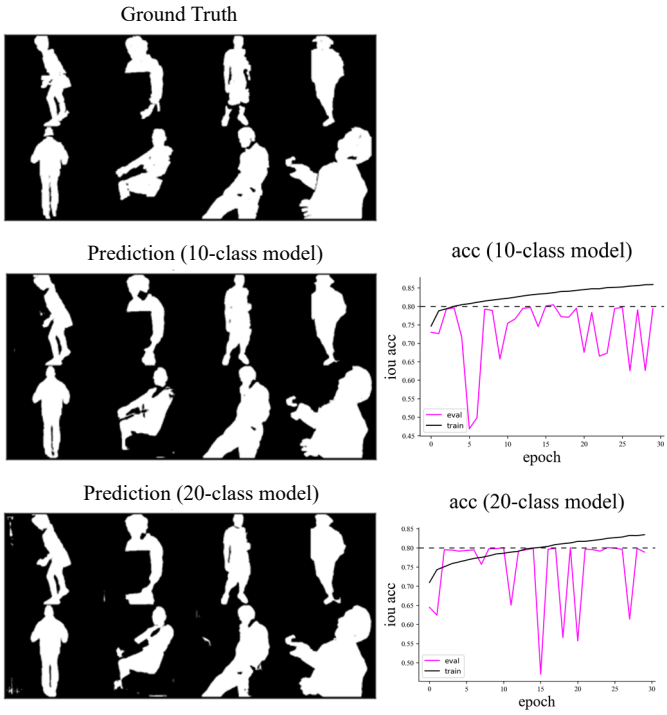
2.3 Apply depthwise-separable conv

I applied depthwise-separable conv to U-Net by replacing the ordinary conv in the contracting path. The original U-Net has 28957716 parameters and is around 7.4 times bigger than the new version, which has 3925816 parameters.

Under the same conditions, training the new version is around 18% faster than training the original one, which, however, is far from my expectation of a 7× speedup, and the reason for this discrepancy is to be found. Moreover, depthwise-separable conv makes the new version deeper, which produces more intermediate variables and requires more cuda memory.

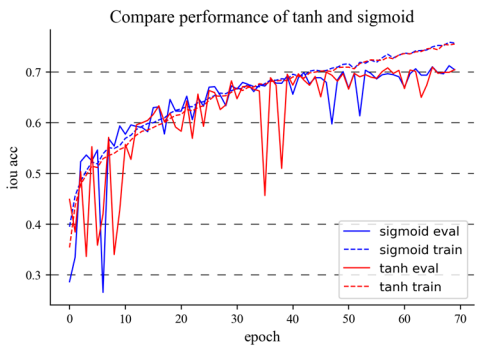
2.4 Improve 2-class segmentation result

Observing that this week's model does better than last week's model in distinguishing person from background, I made use of the model for 20-class and 10-class segmentation trained this week. By finetuning the model for 20-class and 10-class segmentation task, MSE loss decreases from 0.09 to 0.045 and eval acc increases from 0.76 to 0.8. Finetuning the 10-class model gives slightly better predictions (fewer white holes in the background) compared to finetuning the 20-class model, and the reason might be that a model inheriting from a 10-class model is less sensitive to different patterns and colors in an image, as well as the noise in the background.



2.5 Try tanh

I tried tanh as the activation of the output layer of the model trained in the last week. Using tanh as the activation, the labels are modified from 0/1 to -1/1 and the threshold is changed from 0.5 to 0. I expected tanh to perform better than sigmoid since a difference of 2 rather than 1 between two labels motivates the model to be more sensitive to different ground truth and thus to be more accurate. In my experiment, however, there didn't appear apparent difference between the performance of two models.



2.6 Fill Hole *(unsolved)*

The fill-hole code I have at hand hasn't been able to selectively fill the mask: It either fills the gap between two legs along with the normal holes, or it doesn't fill the mask at all.

Moreover, I found that after finetuning the model for 10-class segmentation, there are neither many holes in the background to be filled with 0 nor many holes in the person to be filled with 1. Filling the holes does not noticeably increase the accuracy, because it seems that the problem of the model lies in its tendency to neglect the subtle outline. A good solution is to this problem is to increase the image size, and this leads up to the next difficulty.

2.7 448 × 448 *(unsolved)*

A model with input image size of 448×448 is found to perform better in contour identification. As seen in the following images, though the 448×448 prediction hasn't differentiated the person from the background and segmented different body parts accurately, it identifies the shape of the person's hand at an earlier stage than the 224×224 one. However, since the limitation of cuda memory requires a smaller model, which in my case hurts performance, and a batchsize of 8, which slows down the training, I did not continue with the size of 448×448 after a trial of 30 epochs.

2.8 Several tips

putpalette

Since using continuous colormaps in matplotlib may lead to vague differences in appearance of different classes, it's better to choose discrete colormaps featured with color blocking. Alternatively, the *putpalette* method of an image (of mode L or P) is popular for customizing colors for a mask.

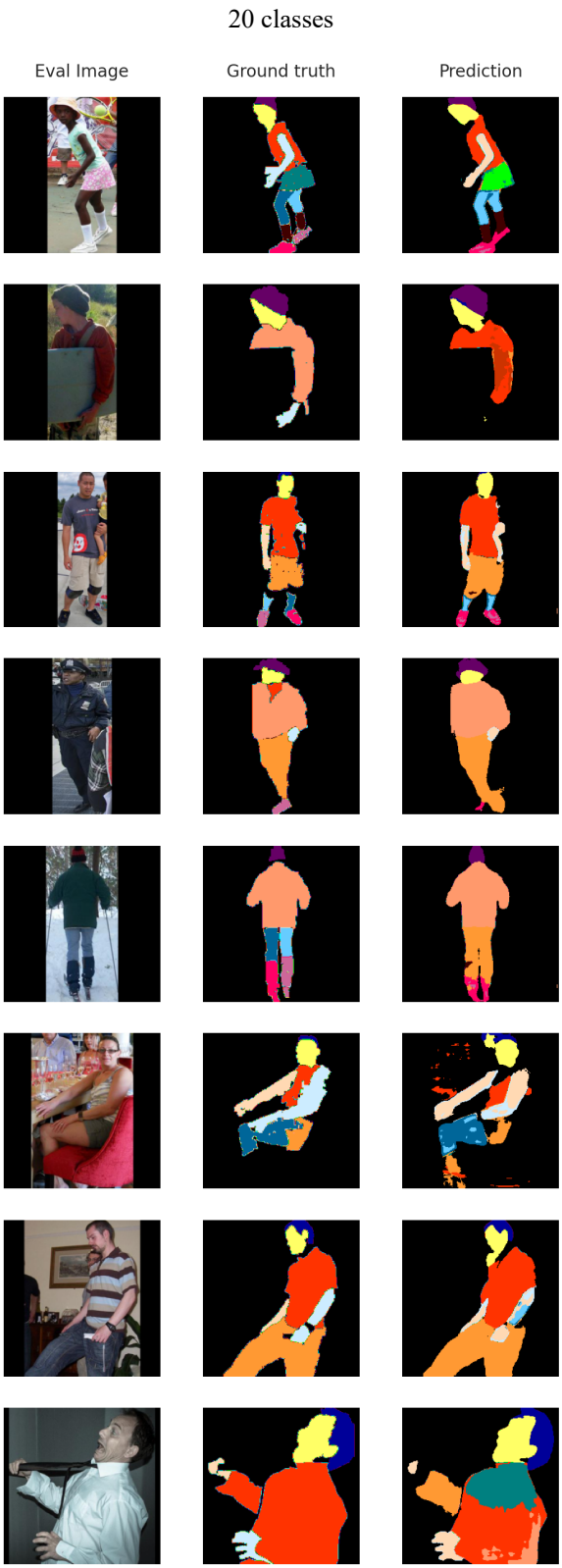
Learn different portions of training set

I tried training the model with a relatively small number of images first and then gradually increase the number of training images. For this strategy, one tip is to avoid always loading images in the same order so that all the images in the whole training set have equal probability of being learned.

Store the data

It is found helpful to store the data of losses and accuracies in a json file for future plotting, which enables more delicate axes modification and data comparison.

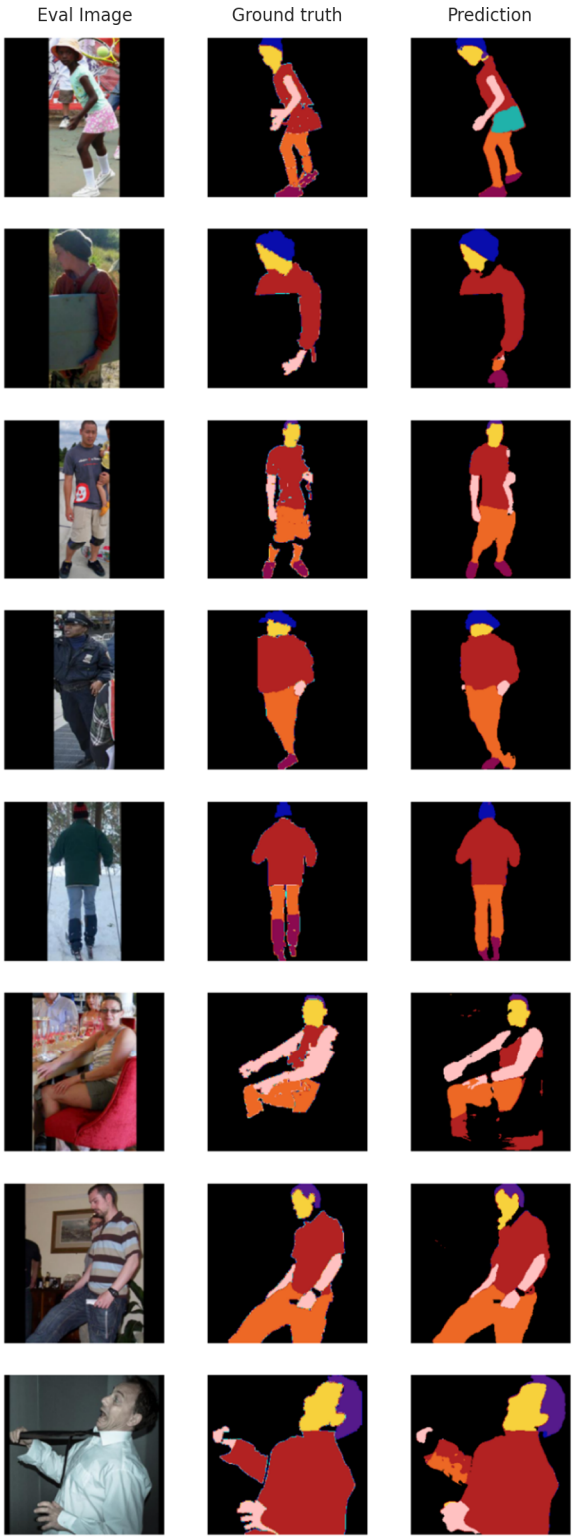
2.9 Demo of performance (eval)



***Comment*:**

As mentioned in 1., the 20-class model has difficulty in distinguishing similar classes such as *UpperClothes* and *Coat*, *LeftLeg* and *RightLeg*, etc. Moreover, some images contain several people and only the person in the front is labeled, and therefore the corresponding predictions tend to have messy background. In this case, I may not consider the messy background as an indicator of flaw of the model.

10 classes



***Comment*:**

Compared to the 20-class model, the 10-class one has much fewer problems in misclassification: The confusing classes are reduced to *Shoes-LowerClothes* and *Hat-Hair*. To improve performance, apart from increasing the training image size, the model needs to be less sensitive to brightness and contrast.